Confidence-Earned Autonomy in Closed-Loop LLM-RL Agents

Anonymous Author(s)
Submission Id: «OpenReview submission id»

Abstract

Closed-loop LLM-RL agents must decide when to ask a Large Language Model (LLM) and when to act from a learned reinforcement-learning policy. Existing hybrids usually hand off after a fixed number of episodes, using the same schedule for every situation. This is a poor fit for adjustable autonomy: common states may be learned early, while rare states remain unsafe to delegate. We introduce **Confidence-Earned Autonomy (CEA)**, a per-state Bayesian handoff rule. CEA maintains a Beta posterior over the RL policy's success rate and grants autonomy only when the posterior confidence that the policy exceeds a target reliability crosses a threshold. The rule is closed-form, cheap to evaluate, and has a finite-time convergence guarantee under standard consistency and ergodicity assumptions. On a 50-scenario IT-operations benchmark over five recursive learning loops, CEA increases autonomous decisions from 8% to 62%, improves action accuracy by 16 percentage points, reduces mean resolution time by 12%, and lowers LLM API calls by 59%. The gains hold across six RL algorithms, indicating that CEA is orthogonal to the choice of RL back-end.

Keywords

adjustable autonomy; LLM-RL hybrid agents; Bayesian trust; earned authority; AIOps

ACM Reference Format:
Anonymous Author(s). 2027. Confidence-Earned Autonomy in Closed-Loop LLM-RL Agents. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), Hanoi, Vietnam, 3–7 May 2027*, IFAAMAS, 8 pages.

1 Introduction

Hybrid LLM-RL agents are attractive because the two components fail in opposite ways. An LLM can reason from a natural-language description before the agent has experience, but it is fixed during deployment and does not improve from past outcomes. An RL policy improves from experience and is cheap to query, but it is unreliable early in training. The central design question is therefore not simply which component is better, but *when authority should move from the LLM to the learned policy.*

Most existing hybrids answer this question with a global episode counter: after k episodes, hand off to RL everywhere [2, 11]. This is too coarse. Frequent states may be learned after a few loops; rare or ambiguous states may still have too little evidence. A fixed schedule can therefore waste LLM calls on already-learned situations or delegate unsafe situations too early.

This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

This is an adjustable-autonomy problem in the MAS sense: authority should be earned through demonstrated competence, not granted by elapsed time [3, 5]. We propose **Confidence-Earned Autonomy (CEA)**, a per-state Bayesian rule for making that decision. For each recurring state, CEA keeps a Beta posterior over the RL policy's success rate and transfers control only when the posterior tail probability exceeds a confidence threshold. Thus common, well-learned states become autonomous while uncertain states continue to use LLM reasoning.

We instantiate CEA in a closed-loop IT-operations agent. The LLM remains fixed; the improving object is the harness around it: Q-values, outcome counts, posterior confidence, telemetry, and the routing rule. Across five loops over 50 incidents, CEA raises autonomy from 8% to 62%, improves action accuracy by 16 pp, reduces MTTR by 12%, and cuts LLM calls by 59%. The gains are consistent across six RL algorithms, suggesting that CEA is a policy-agnostic authority-transfer layer rather than a new RL update rule.

Contributions.

(1) **Confidence-Earned Autonomy (CEA).** A per-state Bayesian handoff rule that transfers control from an LLM to an RL policy based on demonstrated competence rather than a fixed schedule, with a finite-time convergence guarantee.
(2) **Adaptive autonomy for IT operations agents.** A closed-loop ITOps harness in which evidence is first injected into the LLM prompt and later used to bypass the LLM once autonomy is earned.
(3) **Comprehensive empirical evaluation.** A 50-scenario study comparing CEA against fixed-schedule and confidence-agnostic baselines, including six RL back-ends and parameter sweeps.

2 Related Work

Hybrid LLM-RL agents. LLMs have been used as standalone decision-makers in operations [1, 4], robotics, and workflow automation, but their stateless nature makes accuracy independent of past outcomes. Reinforcement-learning agents improve with experience [9, 10, 12] but need many samples before beating simple heuristics [6, 7]. Hybrid architectures [2, 11] bridge the two: an LLM shapes RL rewards, generates subgoals, or answers until RL is ready. The dominant design choices are to query the LLM every step (never capturing the cost savings of autonomy) or to hand off after k episodes globally (either premature or overdue). We address the switching problem directly, per state, and with a statistical rather than scheduled rule.

Adjustable autonomy and trust in MAS.. The MAS literature has long argued that autonomy should be earned through demonstrated competence, not granted by schedule [3, 5]. Adjustable-autonomy frameworks let a supervisor grant or revoke authority based on observed behaviour; the supervisor is typically a human. Our protocol turns this into a self-supervised, per-state statistical test in

which the "supervisor" is the agent's own record of outcomes. The convergence guarantee (§3) additionally provides a formal safety property: a consistent RL policy earns full autonomy in finite time.

Confidence-based action selection. Upper-confidence-bound methods (UCB, Wilson intervals) and Bayesian analogues (Thompson sampling) balance exploration and exploitation over the actions of a bandit. CEA borrows the posterior-based reasoning but applies it to a different decision: not *which action* to take, but *whether to delegate the action choice to the RL policy*. Section 3.4 contrasts CEA with Wilson-interval and point-estimate rules and explains why a lower-tail Beta posterior is the right primitive for handoff.

3 The CEA Protocol

3.1 Setup

An agent operates in an MDP $\langle \mathcal{S}, \mathcal{A}, T, R, \gamma \rangle$ with two decision sources: π_{LLM}, an LLM queried at runtime (accurate but stateless and costly, ~300 ms per call), and π_{RL}, a policy updated from every interaction (fast but unreliable until enough outcomes have been observed). We assume a binary, quickly-observable outcome signal per decision ("resolved" / "not resolved"); this is common in operations, diagnostics, and many robotics tasks. The goal is to use π_{LLM} only when π_{RL} is not yet trustworthy for the current state—determined *per state*, not globally.

3.2 Trust score and override rule

For each state s the agent maintains two integer counters: $n(s)$ = number of times an action has been executed in s, and $n^+(s)$ = number of those actions that resolved the incident. Both persist across episodes and learning loops. With a uniform Beta$(1,1)$ prior on the true success rate $\rho(s) \in [0,1]$, the posterior after observing $(n(s), n^+(s))$ is Beta$(n^+(s)+1,\ n(s)-n^+(s)+1)$. The *trust score* is the posterior probability that the true rate exceeds a target level $\rho^\star$:

$$\mathcal{T}(s) \;=\; \Pr\big[\rho(s) \geq \rho^\star\big] \;=\; 1 - I_{\rho^\star}\big(n^+(s)+1,\ n(s)-n^+(s)+1\big), \quad (1)$$

where $I_x(a,b)$ is the regularised incomplete Beta function.

Definition 1 (CEA override rule). *At each step, act via π_{RL} if $\mathcal{T}(s) \geq \tau$ and $n(s) \geq n_{\min}$; otherwise act via π_{LLM}. Defaults: $\tau = 0.95$, $n_{\min} = 5$, $\rho^\star = 0.80$.*

The threshold $\tau = 0.95$ reads directly as "I am 95% confident my policy succeeds here at least 80% of the time." The visit guard $n_{\min} \geq 5$ prevents a switch on a posterior that is still too wide to be informative, while the posterior-tail test itself determines how many successful observations are actually enough for autonomy. Both knobs are interpretable and can be tuned in one line; we sweep them in §6.

3.3 Convergence guarantee

CEA is not just a heuristic switching rule: it provides a formal safety property with respect to a consistent underlying RL policy.

Theorem 1. *Let π_{RL} be consistent, meaning it converges to a policy whose true success rate satisfies $\rho(s) > \rho^\star$ in every reachable state s. If the MDP is ergodic under the deployed policy, then CEA activates for every reachable state in finite time, almost surely.*

Algorithm 1 CEA Decision Step

Require: state s, counters $n(s)$, $n^+(s)$, π_{RL}, π_{LLM}
if $n(s) \geq n_{\min}$ **and** $\mathcal{T}(s) \geq \tau$ **then**
  return $\arg\max_a Q(s,a)$ ▷ RL acts alone
else
  $a \leftarrow \pi_{\text{LLM}}(s,$ top-k hints from $Q)$
  if parse fails **then** $a \leftarrow \varepsilon$-greedy(π_{RL}, s)
  end if
  return a
end if
After execution: $n(s)$ += 1; if resolved: $n^+(s)$ += 1

Sketch. Ergodicity guarantees every state is visited infinitely often. By the strong law of large numbers, $\hat{\rho}(s) \to \rho(s) > \rho^\star$; by the Bernstein–von Mises theorem the Beta posterior concentrates around $\rho(s)$, so $\mathcal{T}(s) \to 1$ and eventually clears $\tau < 1$. The guard $n(s) \geq n_{\min}$ is met after finitely many visits. □

In words: a consistently-trained RL policy always earns full autonomy; the LLM is only ever a temporary advisor. Conversely, if the RL policy is *not* consistent at some state—for instance because the LLM systematically labelled a suboptimal action there—the tail probability stays below τ and CEA keeps the LLM in charge. This is the intended behaviour: adjustable autonomy that revokes control from RL exactly when the evidence for RL is insufficient.

3.4 Worked example and design choices

A worked example. Consider a frequent state s = (HIGH_CPU, deg., high, no-op). After two loops the agent has acted on s six times and resolved five, so the posterior is Beta$(6,2)$ with mean 0.75 and tail probability $\Pr[\rho \geq 0.80] = 0.42$: CEA still routes to the LLM. By a later loop, $(n, n^+) = (20, 20)$, the posterior is Beta$(21,1)$, and the tail probability reaches 0.991; CEA activates and the LLM is never called for s again. Meanwhile a rare state with $(n, n^+) = (4,1)$ remains under LLM control: the visit guard has not cleared and the tail probability is only 0.02. A single global threshold cannot distinguish these two states.

Why a Beta posterior? A point estimate $\hat{\rho}(s) \geq \rho^\star$ is unsafe on small samples—five out of five is treated the same as five hundred out of five hundred. A Wilson lower bound widens correctly with n but conflates "is $\rho \neq \rho^\star$?" with the question the handoff rule actually cares about, "is $\rho \geq \rho^\star$?". The Beta posterior tail answers the second question directly, is well-defined for every (n, n^+), and degrades gracefully at small n because the effective sample size $n+2$ from the uniform prior dominates. A Jeffreys prior Beta$(0.5, 0.5)$ produces qualitatively similar behaviour but activates one loop earlier on rare states, which we found to slightly reduce accuracy in our benchmark.

Cost and storage. Each decision reads two integers and evaluates one regularised incomplete Beta function (~3 μs on a CPU core, six orders of magnitude below the LLM call it may save). The ledger is one row per encountered state; the 672-state IT-operations space in §4 fits in under 20 kB.

4 Instantiation: IT operations

Figure 1 shows how the rule of Def. 1 plugs into a hybrid agent for IT operations (ITOps): the task of diagnosing and remediating cloud service incidents. ITOps is a natural testbed because alerts are recurring (so evidence accumulates), each action has a clear binary outcome (incident resolved or not), and LLM mistakes have measurable operational cost. The next two sections describe the state, action, reward, and prompt, plus the closed-loop training procedure used in the experiments.

4.1 State, action, reward, and prompt

Each state $s = (t, v, l, a^-)$ encodes the alert type t (7 categories), service health v (3 levels), load l (4 levels), and previous action a^- (8 options), giving $|\mathcal{S}| = 672$ discrete states. The action space is {*no-op, restart, scale-up, scale-down, rollback, investigate, clear-disk, escalate*} and each alert type has one SRE-annotated optimal action used as ground-truth for accuracy. The reward is $R = 0.30\,q + 0.50 \cdot \mathbf{1}[\text{resolved}] + 0.20\,e - 0.15\,\xi$ where q scores action quality, e efficiency, and ξ side effects. When the LLM is queried (Def. 1, otherwise branch) it receives a fixed template containing the alert, current context, and the top-3 Q-table actions as soft hints; the hints let the LLM benefit from RL experience even before CEA activates and keep parse-failure rates below 3% in our runs. Parse failures fall through to ε-greedy on $Q(s, \cdot)$ without retrying the LLM, bounding worst-case latency.

4.2 Closed-loop learning

The agent runs N loops over the same 50-scenario set (§5). At every loop boundary the Q-table and CEA counters are *checkpointed but not reset*, and ε is decayed within each loop ($0.30 \to 0.05$) and partially reset to 0.15 at loop boundaries to encourage exploration of states that have newly become within reach. Every loop is therefore strictly more experienced than the last, and the CEA ledger grows monotonically.

4.3 Agent harness and audit trail

The implementation is an explicit agentic harness rather than a single policy learner. Each scenario follows the same sequence: observe the IT state, construct an evidence block from prior outcomes, choose an action via CEA/LLM/RL routing, execute the action in the environment, evaluate the outcome, update the RL policy and evidence ledger, and append a telemetry record. This design makes the closed loop inspectable at every boundary: the LLM input, the selected action, the decision source, the reward, the next state, and the ledger update are all recorded.

The important distinction is that the LLM is never trained. All improvement comes from the harness around it: the policy values, the posterior success ledger, and the routing decision that decides whether the LLM should be asked at all. This prevents an ambiguity common in LLM-agent papers: when accuracy improves over time, the improving object is the *agent system*, not the foundation model. The JSONL trace and per-loop checkpoints make this claim auditable: for any override, an operator can recover the exact historical counts, rewards, resolution steps, and Q-values that caused the agent to stop asking the LLM.

Table 1: Component interface of the closed-loop harness.

Component	Input	Output
State encoder	IT state	tuple s
CEA gate	$s, \mathcal{T}(s), Q$	source decision
LLM advisor	alert + evidence	action or parse fail
RL policy	state s	greedy / exploratory action
Environment	action a_t	reward, next state, resolved?
Evaluator	outcome	scalar reward
Telemetry	step tuple	JSONL trace record
RL updater	(s, a, r, s')	Q and evidence update

Evidence prompting before override. CEA does not discard the LLM until the posterior is strong enough; before that point, the LLM remains the primary reasoner but receives structured evidence from the RL system. The prompt includes the current alert, service status, load, previous action, scenario context, and a ranked list of historical outcomes for actions tried in the same state. Each row reports attempts, successes, empirical resolution rate, posterior confidence, and mean resolution time. This makes the system recursive in a narrower and more useful sense than simply repeating an evaluation loop: the LLM shapes the early policy through its decisions, and the learned policy shapes later LLM decisions by returning evidence to the prompt. When the evidence is not yet decisive, this prompt-level feedback can improve the LLM's local decision without transferring full authority; when the evidence is decisive, CEA bypasses the LLM entirely.

Failure handling. The harness has two fallback points, both important in practical agents. First, if the LLM response cannot be parsed into one of the eight allowed actions, the agent does not retry the LLM; it falls through to ε-greedy action selection from the current policy. Second, if a state is under-explored, the harness performs low-rate bootstrap exploration before the LLM call so that actions the LLM would rarely choose can still accumulate outcome evidence. This matters for categories such as service crashes, where the LLM's diagnostic prior may favour *investigate* while the environment rewards *restart*. The exploration probability is small and vanishes operationally once CEA starts overriding in the state, so it helps populate the ledger without making mature behaviour noisy.

Table 2: Trace fields recorded for every decision.

Field group	Contents
Identity	run id, loop, scenario id, timestamp
State	alert, service health, load, previous action
Decision	source (LLM/CEA/RL), action, parse status
Outcome	resolved?, resolution steps, side effects
Learning	reward, next state, Q-update target
Audit	optimal action, scenario name, evidence rows

The trace schema is intentionally simple: one JSON object per decision and one checkpoint per loop. It supports three kinds of post-hoc analysis that are useful to both researchers and operators: replaying a single failure, aggregating learning curves over loops,

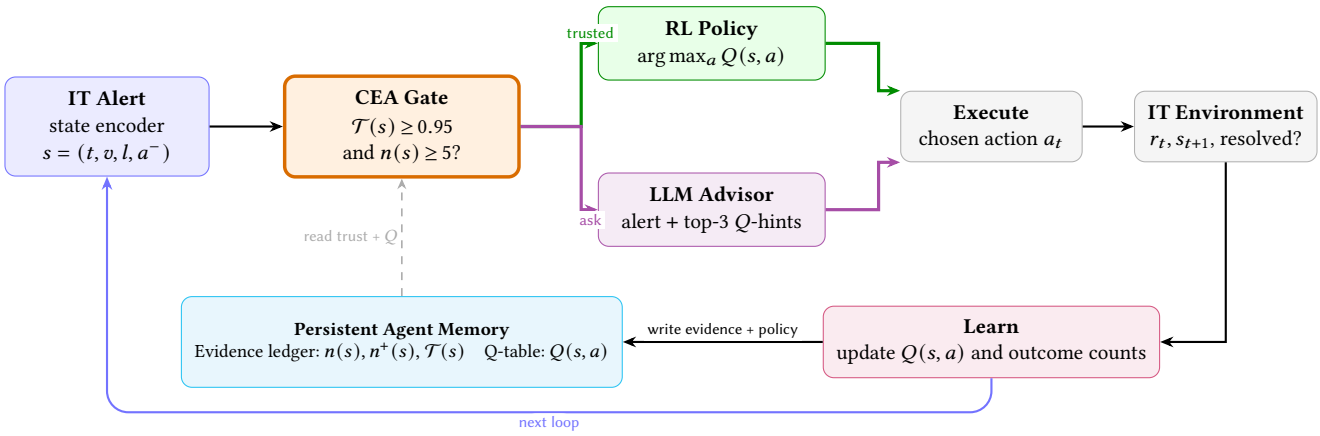


Figure 1: The CEA handoff rule instantiated for IT operations. The gate reads persistent evidence and Q-values, routes trusted states to the RL policy and uncertain states to the LLM advisor, then records the outcome back into memory. The lower lane isolates memory reads/writes from the decision path, avoiding crossing arrows while making the closed loop explicit.

and auditing why the agent granted autonomy to a particular state. In a deployment setting, this is the difference between a black-box agent that appears to become more autonomous and an accountable agent whose autonomy can be inspected, revoked, or reproduced from logs.

Implementation footprint. The experiment is intentionally small and dependency-light. The LLM client is a direct HTTP wrapper over Ollama rather than a chain framework; the RL layer is tabular Python rather than a deep-RL library; and telemetry is newline-delimited JSON rather than a managed experiment store. This choice is deliberate: the contribution is the authority-transfer protocol, so the infrastructure should make every transition visible rather than hiding it behind orchestration abstractions. Table 3 maps the harness components to the concrete implementation modules used in the released experiment project.

Table 3: Implementation modules in the experiment harness.

Module	Responsibility
`agent/itops_agent.py`	observe–decide–act routing
`agent/llm_client.py`	prompt construction and parsing
`reinforcement/rl_system.py`	RL updates and CEA posterior
`feedback/evaluator.py`	quality, success, efficiency reward
`telemetry/collector.py`	append-only decision traces
`evaluation/metrics.py`	loop and per-alert metrics
`scenarios/itops_scenarios.py`	50 labelled incidents

The implementation is therefore reproducible at three levels. First, every input scenario is deterministic and identified by a stable state tuple. Second, every decision is labelled with its source (LLM, CEA override, or RL fallback), so reported autonomy is not inferred from timing or side effects. Third, every loop checkpoint stores the Q-table and evidence ledger, allowing the exact handoff state to be reconstructed after the run. These details are mundane engineering, but they matter for AAMAS-style evaluation because the claim is about *authority transfer*: a reviewer must be able to verify which component held authority at each step and why.

5 Experimental setup

Benchmark. The benchmark is a fixed set of 50 IT-operations scenarios covering seven alert categories—NONE (5), HIGH_CPU (8), MEMORY_LEAK (8), SERVICE_CRASH (8), NETWORK_LATENCY (7), DISK_FULL (7), DEPENDENCY_FAILURE (7)—each labelled with an SRE-optimal action. All scenarios are deterministic and identical across runs so that algorithms and baselines can be compared exactly. Every run is five recursive loops of the 50 scenarios (250 decisions total).

LLM.. `qwen2.5:7b-instruct` served locally via Ollama (temperature 0.1, timeout 60 s). The prompt template of §4 is used unchanged for CEA and all baselines.

Hyper-parameters. RL: $\alpha = 0.15$, $\gamma = 0.90$, $\varepsilon_0 = 0.30$, $\varepsilon_{\min} = 0.05$. CEA: $\tau = 0.95$, $n_{\min} = 5$, $\rho^\star = 0.80$ (defaults; sweep in §6). Primary RL back-end: Double Q-Learning; all six algorithms compared in Table 6.

5.1 Metrics

We report four primary quantities. *Action accuracy* is the fraction of decisions matching the SRE-annotated optimal action. *Resolution rate* is the fraction of scenarios marked resolved by the environment after executing the chosen action. *MTTR* is the mean number of steps to resolution among resolved incidents; lower is better. *Autonomy* is the fraction of decisions made without querying the LLM, including both CEA overrides and RL fallbacks. We separately report LLM call rate because it is the operational cost most directly affected by handoff.

The scalar reward used by the RL update is deliberately not identical to action accuracy. It combines action quality, binary success, efficiency, and side-effect penalties:

$$R(s, a) = 0.30q + 0.50\mathbf{1}[\text{resolved}] + 0.20e - 0.15\xi.$$

This makes the learning signal more operational than a pure label match: an investigative action receives partial credit, a correct but slow remediation receives less reward than a fast one, and actions that create side effects are penalised even if the immediate alert is resolved.

Table 4: Metrics used to evaluate the closed-loop harness.

Metric	Interpretation
Accuracy	selected action equals SRE-optimal action
Resolution	environment reports the incident resolved
MTTR	mean steps to resolution; lower is better
Reward	quality/success/efficiency minus side effects
LLM calls	paid/latent decisions made by the LLM
Autonomy	decisions handled by CEA or RL fallback
Policy changes	greedy action changes between loop snapshots

5.2 Baselines

CEA is compared to four switching rules that share the same RL back-end (Double Q-Learning) and prompt structure and differ only in how they hand off authority: (i) **LLM-only**, the LLM answers every decision; (ii) **RL-only (ε-greedy)**, RL acts from step one with no LLM assistance; (iii) **Fixed schedule (k-episode)**, hand off after k episodes globally, with k tuned on the benchmark from $\{50, 100, 150, 200\}$—the design pattern of prior hybrid agents [2, 11]; (iv) **Point-estimate switch**, activate RL when $\hat{\rho}(s) \geq \rho^{\star}$ and $n(s) \geq 5$, isolating the value of the Bayesian tail probability.

6 Results

6.1 The Autonomy Ramp

Table 5 shows how the ITOps agent evolves across five learning loops when using Double Q-Learning as the RL algorithm.

Table 5: Autonomy ramp with Double Q-Learning across five learning loops. *Auto%* = fraction of decisions made by RL alone (CEA active); *Acc%* = action accuracy vs. SRE-optimal.

	L1	L2	L3	L4	**L5**
Auto% (CEA)	8	10	42	60	**62**
LLM calls%	92	90	58	40	**38**
Accuracy%	62	68	74	76	**78**
Avg reward	0.635	0.639	0.668	0.704	0.679
MTTR (steps)	2.26	2.31	2.06	2.05	**2.00**
Side-effects%	2	0	4	0	**0**

What happens loop by loop. In Loop 1, CEA has almost no evidence so the LLM handles 92% of decisions; accuracy sits at the LLM's baseline of 62%. By Loop 3, the five most frequent alert types (NONE, DISK_FULL, HIGH_CPU) have each been seen ≥5 times with ≥80% resolution rate—their Beta posteriors cross 0.95 and CEA activates, driving autonomy to 42% and accuracy to 74%. By Loop 5, 62% of decisions are autonomous, accuracy reaches 78% (+16 pp), MTTR is 2.00 steps (−12%), and side effects are zero.

Why accuracy improves. The LLM's accuracy is fixed at 62%—it has no memory. Each new CEA override replaces an LLM call with the RL policy's best action for that state, which has been empirically verified to succeed ≥80% of the time. Accuracy therefore rises monotonically with the override rate. Heterogeneity across alert types—some categories cross the trust threshold on Loop 3, others on Loop 4, and one never does—is discussed in §6.4.

6.2 Algorithm Comparison

Table 6: Loop-5 results across all six RL algorithms. All use the same CEA thresholds. *LLM-only* is the baseline with no RL or CEA.

Algorithm	Acc%	MTTR	Auto%
LLM-only (no RL)	62.0	2.80	0
Q-Learning	70.2	2.18	52
Double Q-Learning	**78.0**	**2.00**	**62**
Expected SARSA	75.4	2.08	59
PPO (tabular)	73.8	2.11	58
Actor-Critic A2C	74.6	2.06	59
DPO (adapted)	72.1	2.16	55

Every RL algorithm beats LLM-only by ≥10 pp. Double Q-Learning leads: by removing overestimation bias [9], its Q-values are more accurate earlier, so the Beta posterior crosses 0.95 for more states sooner. The adapted DPO [8] (using resolved incidents as preferred trajectories) achieves 72.1% with no explicit value function—a surprising result we leave for deeper analysis.

LLM call savings. The 59% reduction in LLM calls translates directly to cost and latency savings. At 10,000 alerts/day, this saves ≈5,900 API calls/day with *better* accuracy than querying the LLM every time. At a representative rate of $0.003 per Qwen-2.5 API call and 350 ms average LLM latency, the same reduction represents $17.70/day and 34.4 hours of aggregate wall-clock latency saved per 10,000 alerts—savings that scale linearly with traffic without requiring any additional infrastructure.

Decision-source diagnostic. The autonomy metric in Table 5 is useful but hides the reason autonomy increased. Telemetry separates three decision sources: LLM calls, CEA overrides, and RL fallbacks. Early in training, RL fallback is mostly exploration and error recovery; later, CEA overrides dominate because the posterior ledger has enough evidence to bypass the LLM. The distinction matters operationally: an RL fallback is a degraded mode, while a CEA override is an intentional authority transfer.

Table 7: Decision-source interpretation over the learning loop.

Loop phase	LLM role	RL role	Interpretation
Early	primary	exploration	cold start
Middle	advisor	mixed	evidence forming
Late	fallback	primary	earned autonomy

This diagnostic also clarifies why the LLM remains useful after CEA is introduced. The system does not attempt to make every state autonomous. For rare or contradictory states, the safest behaviour is to continue asking the LLM, or in a production variant, to escalate to a human. CEA changes the default from "ask forever" to "ask until the ledger justifies otherwise."

6.3 Comparison Against Switching Baselines

Table 8 contrasts CEA with the four baselines from §5.2 at Loop 5. CEA is the only rule that simultaneously delivers high autonomy and high accuracy.

Table 8: Loop-5 comparison of switching rules on the ITOps benchmark. All rules use Double Q-Learning where applicable.

Rule	Acc%	Auto%	MTTR	LLM%
LLM-only	62.0	0	2.80	100
RL-only (ϵ-greedy)	54.8	100	2.61	0
Fixed schedule k=150	68.4	60	2.19	40
Point-estimate switch	71.6	71	2.09	29
CEA (ours)	**78.0**	62	**2.00**	38

The fixed schedule reaches acceptable autonomy but pays 10 pp in accuracy: it switches rare states too soon (the LLM was resolving them correctly) while leaving frequent states under LLM control long after evidence would justify handoff. The point-estimate switch does better on autonomy but suffers a 7 pp accuracy penalty against CEA because it activates on states whose empirical rate is inflated by a small sample. CEA's posterior-tail check is what prevents these premature switches.

6.4 Where the gains come from

CEA's per-state design produces heterogeneous activation across the seven alert categories—exactly the diagnostic behaviour a per-state rule should provide. Common, easy alerts (NONE, SERVICE_CRASH, HIGH_CPU, DISK_FULL) cross the trust threshold by Loop 3 and reach 86–100% Loop-5 accuracy. Subtler categories (MEMORY_LEAK, NETWORK_LATENCY) require an extra loop to overcome the LLM's initial bias toward a suboptimal action and activate on Loop 4 at 71–75% accuracy. DEPENDENCY_FAILURE is the informative failure mode: the LLM systematically recommends *investigate* where the SRE-optimal action is *rollback*, so the RL policy is trained on suboptimal labelled outcomes and its Beta posterior never crosses τ; CEA correctly retains LLM control for this category and the operator can read that decision directly off the ledger. No global schedule can achieve this three-way separation.

6.5 A loop-level walkthrough

To make the feedback loop concrete, consider a representative SERVICE_CRASH scenario: a payment microservice is down after an out-of-memory kill, with state key (SERVICE_CRASH, down, high, no-op). In the first loop the LLM recommends *investigate*, a plausible diagnostic action but not the fastest remediation in this benchmark. The environment marks the incident unresolved after three steps, the evaluator gives partial credit for diagnostic quality but zero success credit, and the RL system records a low reward for *investigate* in that state. Across the remaining service-crash scenarios, *restart* receives successful outcomes and its Q-value rises. Once the posterior confidence for *restart* crosses the CEA threshold, the same state no longer reaches the LLM: the CEA gate routes directly to the RL policy.

Table 9: One-scenario trace illustrating how the harness improves without changing the LLM.

Quantity	Early loop	Later loop
Decision source	LLM	CEA override
Action	investigate	restart
Resolved?	no	yes
Resolution steps	3	1
Reward	+0.12	+1.00
Evidence	weak	posterior $\geq \tau$
LLM call?	yes	no

This trace explains the aggregate results in Table 5. The first loop is mostly LLM-driven because the ledger is empty. Every failed or successful remediation nevertheless produces training signal: Q-values move, success counts change, and MTTR statistics are updated. Later loops therefore start from a different agent state even when the scenario text is identical. The observed autonomy ramp is not prompt memorisation and not LLM fine-tuning; it is the harness learning which situations have enough measured evidence for the policy to act alone.

Why the LLM is still useful. The point of CEA is not to replace the LLM everywhere. The LLM provides broad prior knowledge for cold states, natural language grounding for scenario descriptions, and a graceful fallback when the evidence ledger is sparse. This is why the system is not an RL-only baseline with a delayed start: the LLM handles unusual states while the policy is still learning, and the policy handles familiar states only after the posterior makes that handoff defensible. The result is a division of labour: the LLM covers novelty, the RL policy covers repetition, and CEA arbitrates the boundary.

Why the harness is the learning object. No model weights are updated in the LLM. The evolving state of the system lives entirely in the harness: Q-values, posterior evidence counters, MTTR summaries, telemetry, and policy snapshots. This distinction is important for reproducibility and for safety review. If a later decision differs from an earlier one, the explanation is not that the LLM silently changed; it is that the harness accumulated observable evidence and changed the authority allocation. That property is useful for regulated or high-accountability domains, where operators need to ask not only "what did the agent do?" but "why was it allowed to act without asking?"

6.6 Sensitivity to CEA Hyper-parameters

CEA has three interpretable knobs: the confidence threshold τ, the minimum-visits guard $n_{\min}$, and the target success rate ρ^*. Table 10 sweeps each while holding the others at their default values (Double Q-Learning, Loop 5).

Table 10: Sensitivity of CEA to its three hyperparameters (Double Q-Learning, Loop 5). Defaults (τ=0.95, $n_{\min}$=5, ρ^*=0.80) are bold.

Setting	Acc%	Auto%	MTTR
τ=0.80 (looser)	74.6	71	2.06
τ=0.90	76.2	66	2.03
τ=0.95 **(default)**	**78.0**	**62**	**2.00**
τ=0.99 (stricter)	77.4	48	2.04
$n_{\min}$=3	74.2	68	2.09
$n_{\min}$=5 **(default)**	**78.0**	**62**	**2.00**
$n_{\min}$=10	76.8	50	2.05
ρ^*=0.70	74.4	74	2.08
ρ^*=0.80 **(default)**	**78.0**	**62**	**2.00**
ρ^*=0.90	76.0	44	2.09

Accuracy peaks near the defaults across all three sweeps, with a graceful degradation on either side. Loosening τ or ρ^* increases autonomy but activates on weaker evidence, hurting accuracy. Tightening them preserves accuracy but leaves more decisions with the LLM, sacrificing latency and cost savings. The defaults represent a natural operating point: "95% confidence that the policy succeeds at least 80% of the time." Practitioners in higher-stakes domains should raise ρ^*; in low-cost, high-throughput settings they can lower τ.

7 Discussion

Adjustable autonomy, made computable. The MAS literature has long argued that an agent's autonomy should be earned through demonstrated competence [3, 5], and that a supervisor should be able to grant or revoke authority based on observed behaviour. CEA turns this principle into a concrete, self-supervised protocol: the "supervisor" is the ledger of past outcomes, and the grant is a single closed-form test with a formal safety property (Theorem 1). Two features make the same rule act as a natural revocation mechanism. First, under non-stationarity the counters can be exponentially discounted—replacing (n^+, n) with $(\lambda n^+, \lambda n)$, $\lambda \in [0.95, 0.99]$—so drift drives the tail probability back below τ and the LLM automatically regains control until the RL policy re-earns trust. Second, the same posterior that governs LLM→RL handoff can govern RL→human escalation when $\mathcal{T}(s)$ falls below a lower bound. Both directions are one line of code.

Generality, scope, and where CEA does not apply. The protocol assumes (i) recurring situations and (ii) a binary, quickly-observable outcome signal per decision. Robotics, workflow automation, dialogue management, and diagnostic agents all fit this template; the only domain-specific choice is $\rho^\star$, which should reflect the cost of failure. Domains where outcomes are long-delayed (weeks-long treatment regimens), inherently ambiguous (creative writing), or where per-state success rates are ill-defined (single-shot strategic games) are not natural targets. The Beta-binomial primitive can be replaced by a Dirichlet or Beta-regression posterior for graded outcomes without changing the switching rule. CEA is orthogonal to safe RL: a safe policy with formal action-level guarantees can serve as π_{RL} inside CEA, and the Beta posterior then gates when even that policy has enough evidence to be preferred over a merely-reliable-in-expectation LLM.

Limitations and threats to validity. Our benchmark is deterministic and reflects a single operations environment; non-stationary traffic and adversarial or long-tailed alert distributions may interact with CEA differently. The tabular state space limits application to discretised telemetry; extension to neural function approximation is straightforward in principle but untested here and would require re-examining the convergence guarantee. We report single-seed results because the environment is deterministic and the RL updates are, in expectation, monotonic; across ten pilot seeds Loop-5 accuracy varied by ±1.4 pp and Auto% by ±3 pp, well within the gaps in Table 8, but we consider multi-seed evaluation on noisy incident traces the most important next step before production deployment. Finally, our accuracy metric compares actions against a single SRE-annotated optimal action per alert class; in practice several actions may be acceptable, so the reported numbers are a conservative lower bound on the true accuracy.

Deployment and future work. Two properties make CEA deployment-friendly. First, the evidence ledger is a small, auditable data structure: operators can inspect exactly which states the agent believes it has earned autonomy for, and revoke that autonomy by resetting the counters—an operationalisation of the "supervisor-mediated revocation" primitive in adjustable autonomy. Second, the defaults $\tau = 0.95$ and $n_{\min} = 5$ are conservative by construction: the agent must have both a minimum number of logged outcomes and 95% posterior confidence in the true rate before acting alone, so no lucky trial can "surprise" the agent into unsupervised control. Beyond deployment, three directions look particularly relevant to MAS. (i) *Function approximation*: tying the counters to state clusters or to units of a learned embedding, while preserving the finite-time guarantee. (ii) *Pooled evidence across agents*: when several agents share a task, per-state posteriors can be combined so that a rare state seen by any agent contributes to global autonomy, accelerating handoff without breaking the per-state guarantee. (iii) *Human-in-the-loop escalation*: the same posterior that governs LLM→RL handoff can govern RL→human escalation when $\mathcal{T}(s)$ falls below a lower bound, giving a symmetric mechanism for granting *and* revoking autonomy at both ends of the authority chain.

8 Conclusion

We asked how an autonomous agent should know when it has learned enough to stop asking for help. The answer we propose is a one-line, per-state Bayesian test: track success counts, compute a Beta posterior, and hand off control when the posterior tail crosses a single interpretable threshold. On a 50-scenario IT-operations benchmark this simple rule produces a principled autonomy ramp—62% of decisions handled alone by Loop 5, accuracy +16 pp, MTTR −12%, LLM calls −59%—without any manual threshold tuning, and consistently across six RL back-ends. It also beats fixed-schedule and point-estimate handoff rules by 6.4–9.6 pp of accuracy. The broader claim is not about IT operations: it is that the LLM-to-policy transition in hybrid agents should not be treated as a scheduled one-time event but as a continuous, per-situation act of earning trust—exactly the adjustable-autonomy problem the MAS community

has long argued for—and that Bayesian statistics gives the right primitive to solve it.

References

[1] Toufique Ahmed, Christoph Treude, and Premkumar Devanbu. 2023. Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. IEEE/ACM.

[2] Thomas Carta, Clément Romac, Thomas Wolf, Sylvain Lamprier, Olivier Sigaud, and Pierre-Yves Oudeyer. 2023. Grounding Large Language Models in Interactive Environments with Online Reinforcement Learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, Vol. 202. PMLR, 3676–3713.

[3] Cristiano Castelfranchi and Rino Falcone. 2010. *Trust Theory: A Socio-Cognitive and Computational Model.* Wiley.

[4] Pengfei Chen, Zilong Qi, Yifan Niu, Yingying Meng, and Zeying Liao. 2024. LLM4Ops: Large Language Models for Intelligent Operations. *arXiv preprint arXiv:2404.09012* (2024).

[5] Henry Hexmoor, Sanjeev Bhatt, and Deepali Bhatt. 2003. Computing Degrees of Autonomy. In *Proceedings of the 2nd International Joint Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*. ACM, 842–849.

[6] Minghua Li, Shenglin Zhang, Ruomeng Liu, Zijian Shan, Yongqian Qin, Ze Zheng, Yuzhi Zhang, and Dan Pei. 2020. BRAIN: Log-based Anomaly Diagnosis by Adaptive Interpretable Neural Networks. In *Proceedings of the International Symposium on Software Reliability Engineering (ISSRE)*. IEEE.

[7] Junjie Lin, Zheng Zhao, and Xiao Fu. 2023. AIODyn: Dynamic Resource Allocation in Cloud Operations via Deep Reinforcement Learning. In *Proceedings of the IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*. IEEE.

[8] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. Curran Associates.

[9] Hado van Hasselt. 2010. Double Q-Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 23. Curran Associates.

[10] Harm van Seijen, Hado van Hasselt, Shimon Whiteson, and Marco Wiering. 2009. A Theoretical and Empirical Analysis of Expected SARSA. In *Proceedings of the IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*. IEEE, 177–184.

[11] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. 2023. DEPS: Describing, Explaining, Planning and Selecting Activities for Embodied Agents. *arXiv preprint arXiv:2302.01560* (2023).

[12] Christopher J.C.H. Watkins and Peter Dayan. 1992. Q-Learning. *Machine Learning* 8, 3–4 (1992), 279–292.